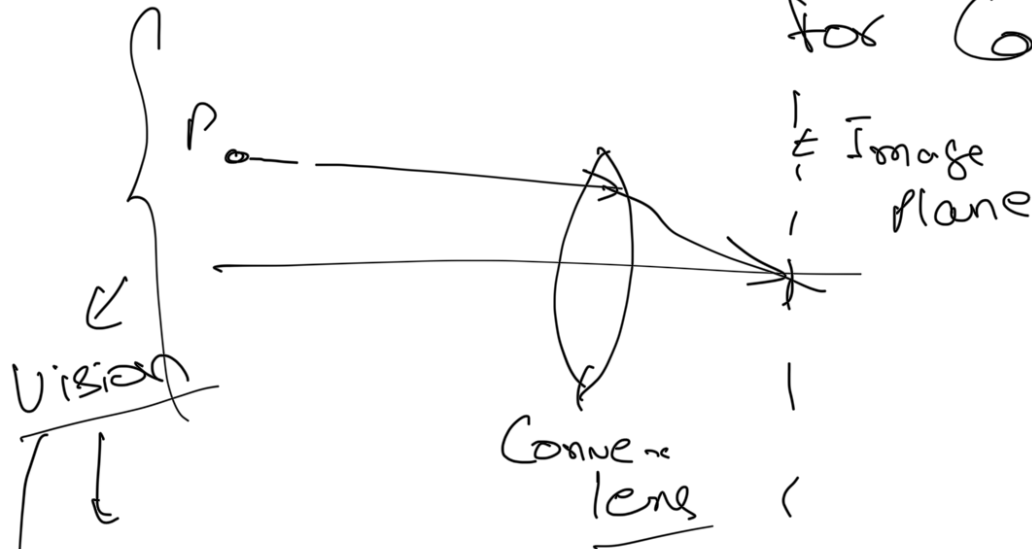


Computer Vision



human like vision
for computers



vision

gives me image

but there is 'no sense'
of the image

Intelligent vision for the computer

dividing into entities (dog vs cat)

1) Object Classification

dividing into groups

2) object 'recognition' (Identification)
→ facebook face tagging

3) Segmentation (Dividing Images)

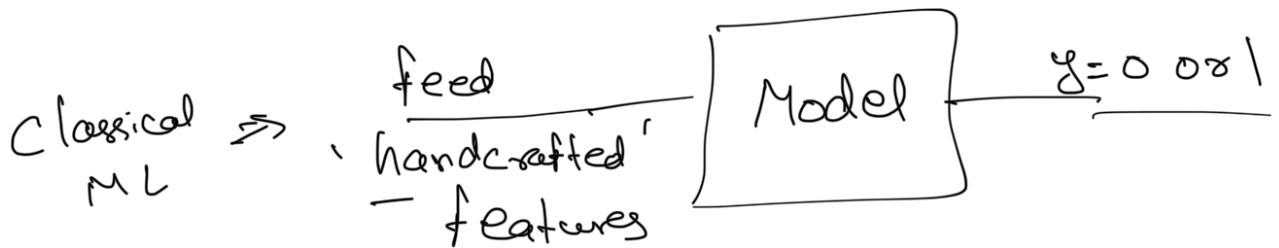
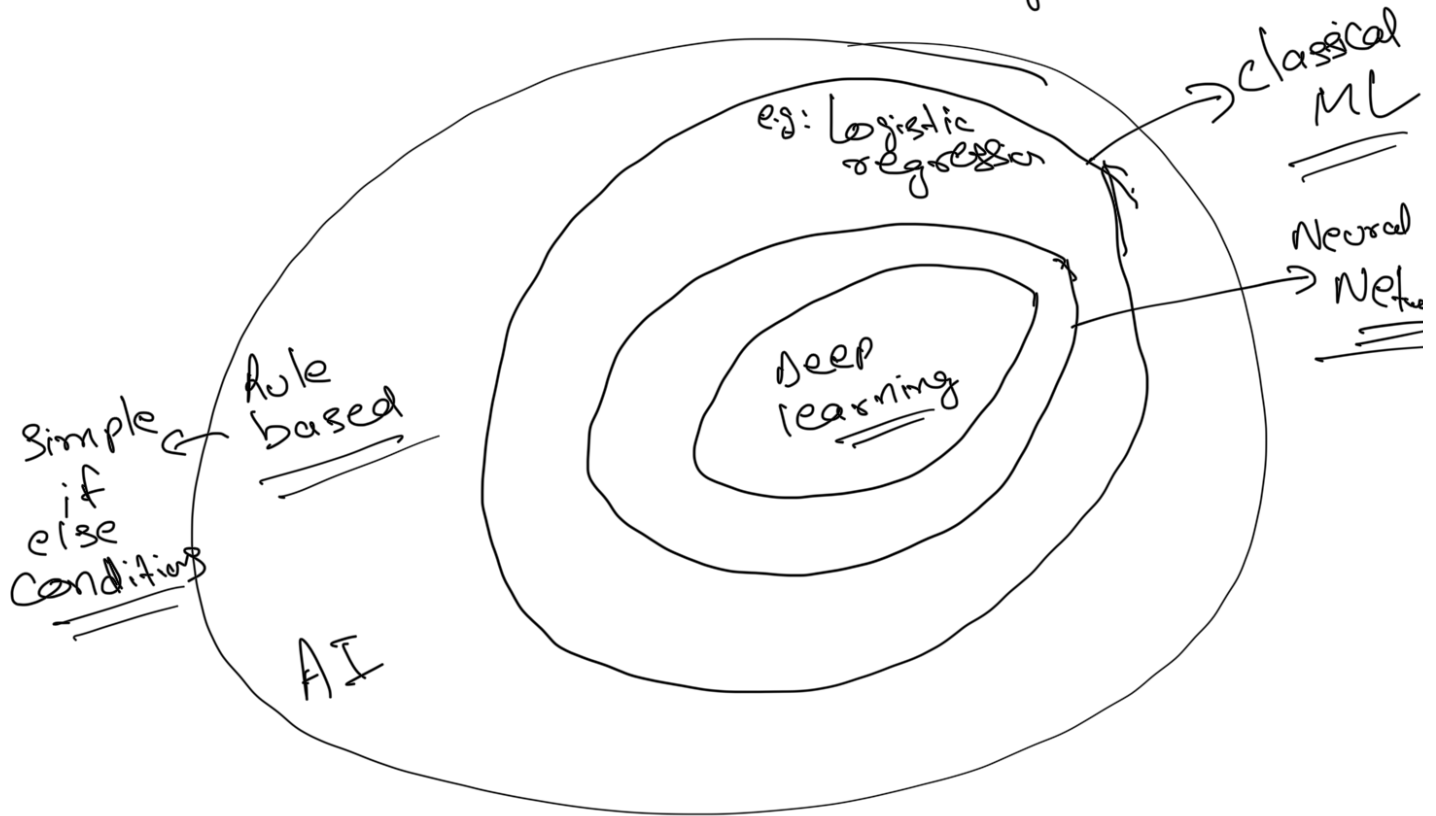
4) 3D Reconstruction
(Depth estimation)

Dividing image into regions

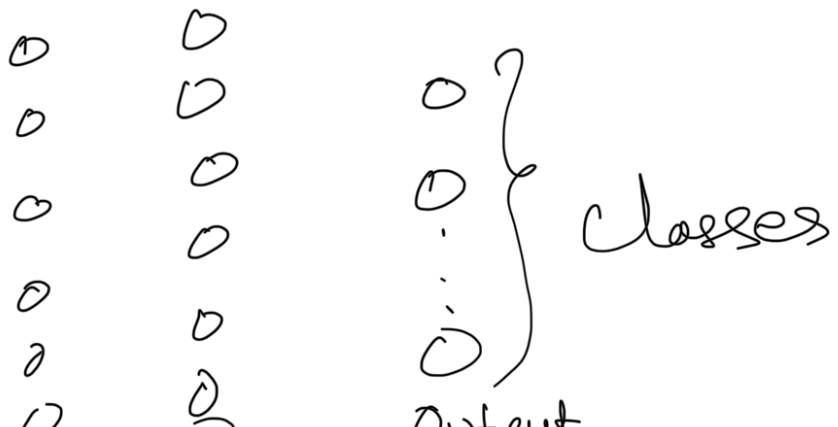


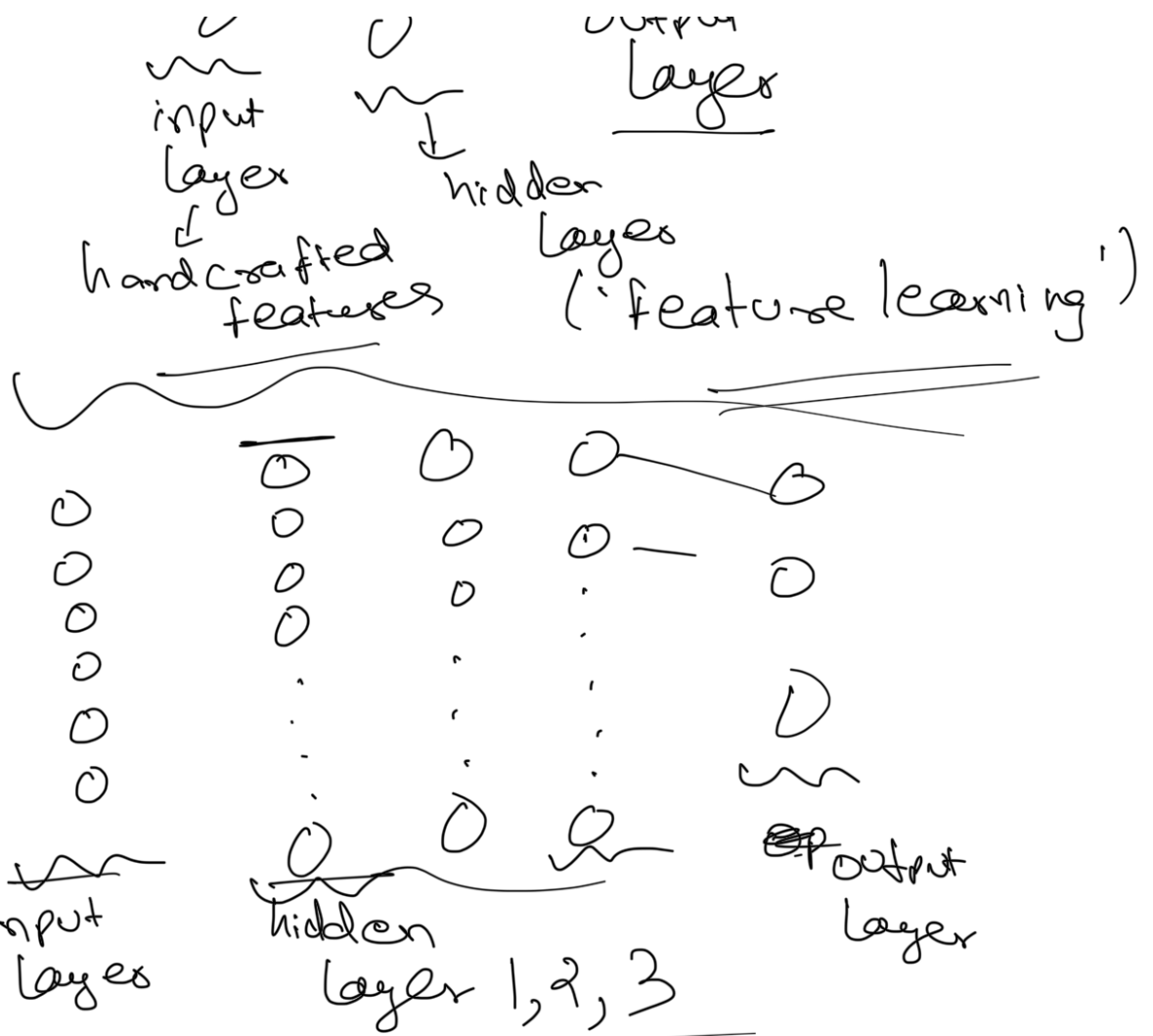
19) 'Flow Identification'

How to get intelligent vision:



Neural Network





not a good idea

as datasets were very small

Situation has changed

Data is in abundance

non-linear function

$$y = f(x)$$

output input

$$y = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$$

$n+1$ points

Deep learning

Why does stacking layers
help in better intelligence

'World is a composition'

Any entity in the physical
world can be said to be
composed of basic entities.

Chemistry: atoms → molecules → lattice →
substance

Music ⇒ 'Sa re ga ma'
7 notes

Images ⇒ pixel → regions → scene

Coordinate system ⇒ points → lines → planes → object
shape

NLP → Character → words → sentences →
Paragraphs → Stories

Neural Network

↓
Composition of Neurons

Composition
of

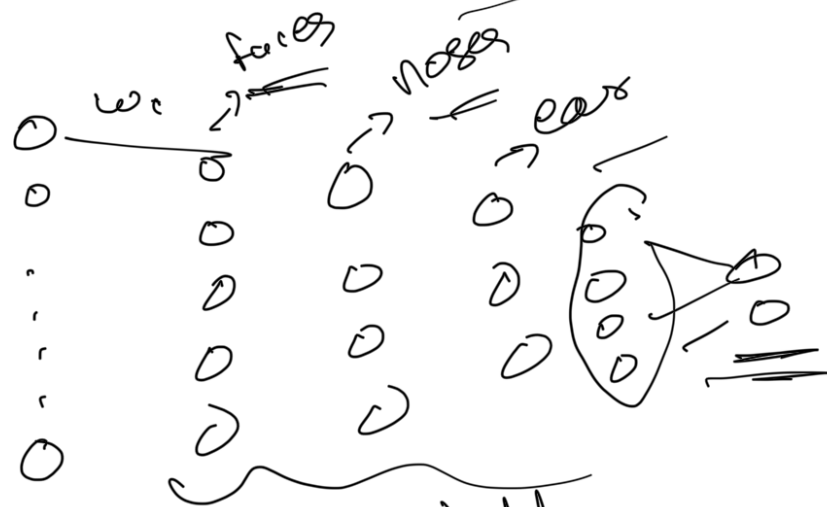
↔

Composition
of

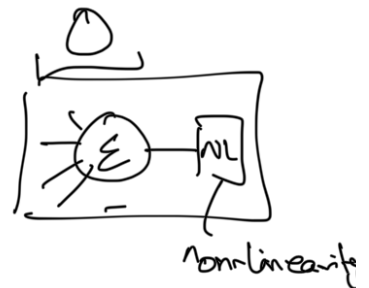
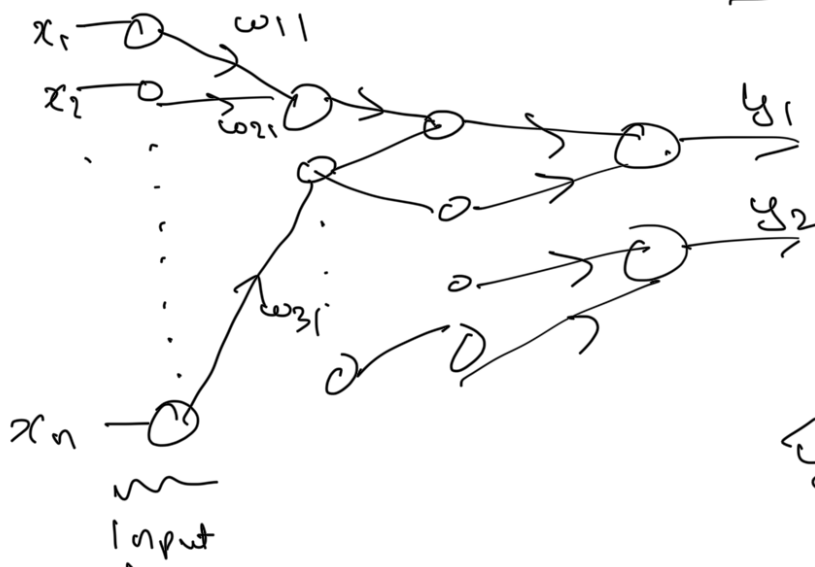
Neuron

Entities in physical world

Computational analogue of real world.



Every hidden layer is trying to find a certain part of composition



$$\hat{y}_1 = f(x_1, \dots, x_n)$$

layer

Training Algorithm

$$J = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_{true})^2$$

Minimize

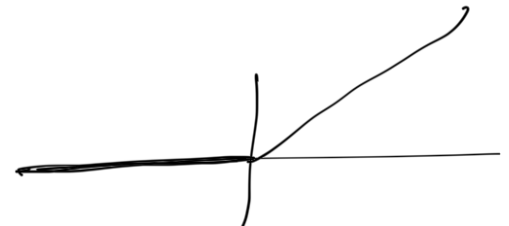
Inference

$$\hat{y}_i \rightarrow \underline{0.8} \rightarrow \underline{\text{Class} = 1}$$

Non-linearities:

- Sigmoid $\Rightarrow z = \frac{1}{1+e^{-x}}$

- tanh

- ReLU $\Rightarrow$ 

- Softmax $\Rightarrow$

Primarily used for the output layer.

Converts outputs to probabilities.

$$z_j = \frac{e^{z_j}}{\sum_{i=1}^N e^{z_i}}$$

$N \Rightarrow \# \text{ of classes}$

$z_i \Rightarrow$ neurons in the output layer

$$x_1 = 0$$

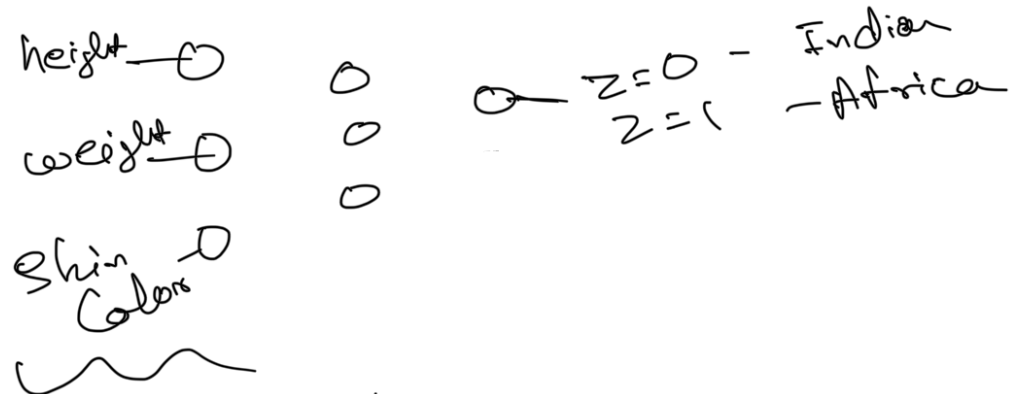
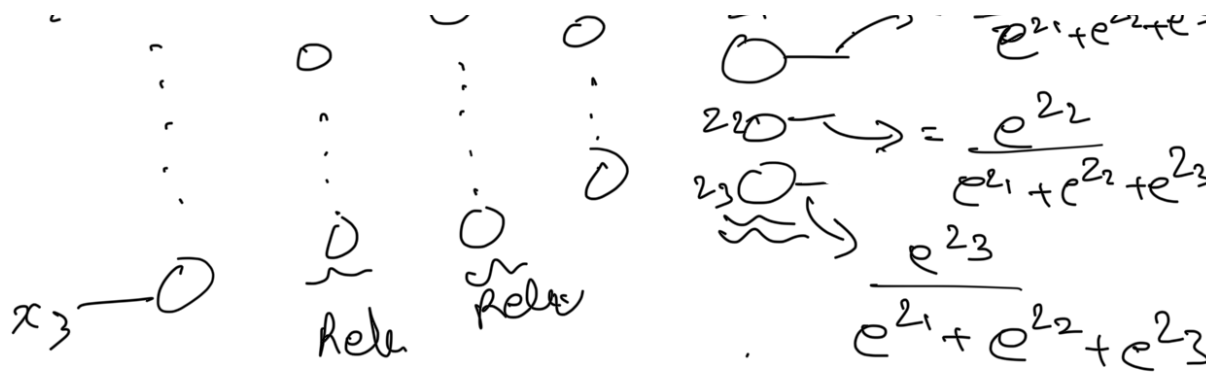
$$x_2 = 0$$

$$0$$

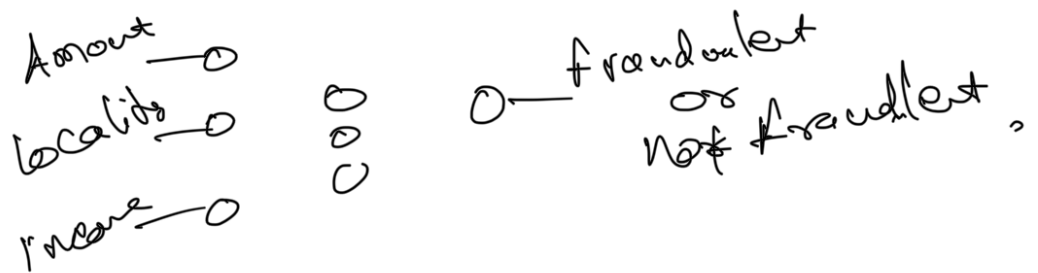
$$n$$

$$z_1$$

$$z_i = \frac{e^{z_i}}{z_1 + z_2}$$



All input features
are independent
of each other.



Simple Neural Network work
well only when the input
features are independent of
each other.

Applying simple neural network to
images.

1 — ac

— u

$x_1 \rightarrow 0$

$x_2 \rightarrow$

0

:

:

:

:

0

0 — G/dog

x_{789} 0

Aim is to get a new neural network model which utilizes the fact that nearby pixels are related to each other.

(also known as convolution)

Spatial filtering

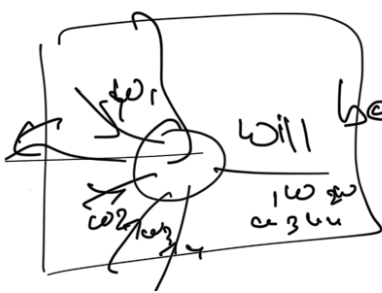
$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 0 & 0 & 1 \\ 0 & -1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

$$f(m) = w_1 p_1 + w_2 p_2 + w_3 p_3 + w_4 p_4 + w_5 p_5 + w_6 p_6 + w_7 p_7 + w_8 p_8 + w_9 p_9$$

$$mask = \begin{bmatrix} w_1 & w_2 & w_3 \\ w_4 & w_5 & w_6 \\ w_7 & w_8 & w_9 \end{bmatrix}$$

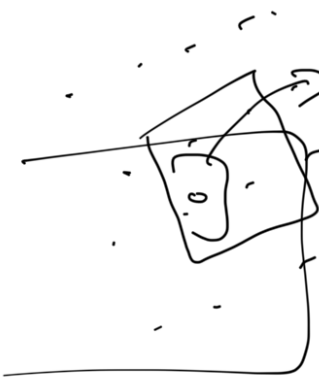
3x3, 5x5, 7x7

will be learned.



basic unit in convolutional

mask

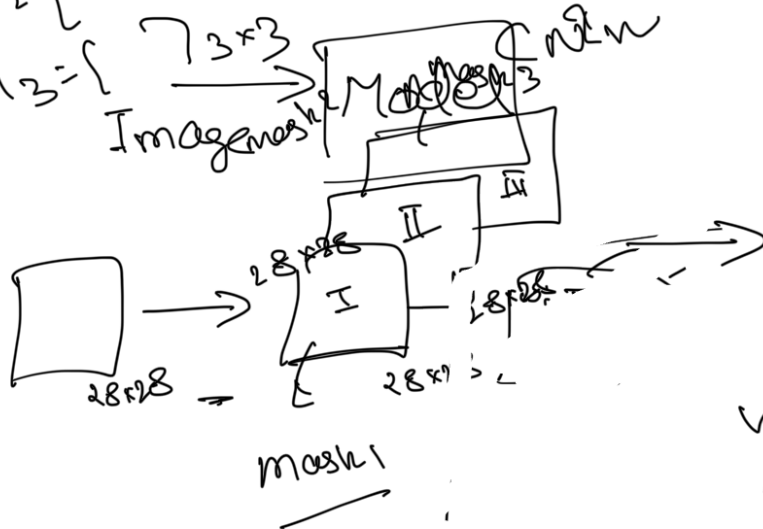


$$M_1 = \begin{bmatrix} \dots \end{bmatrix}_{3 \times 3}$$

$$M_2 = \begin{bmatrix} \dots \end{bmatrix}_{3 \times 3}$$

$$M_3 = \begin{bmatrix} \dots \end{bmatrix}_{3 \times 3}$$

Images $\rightarrow$ Map $\rightarrow$ CNN



With each layer
CNNs are trying
to make pixels
independent $\Rightarrow$

6 masks

but a
feed forward
layer can be
used in
the end

$$J = \frac{1}{m} \sum_{i=1}^m (\hat{y} - y)^2$$

$$y = f(\text{image})$$

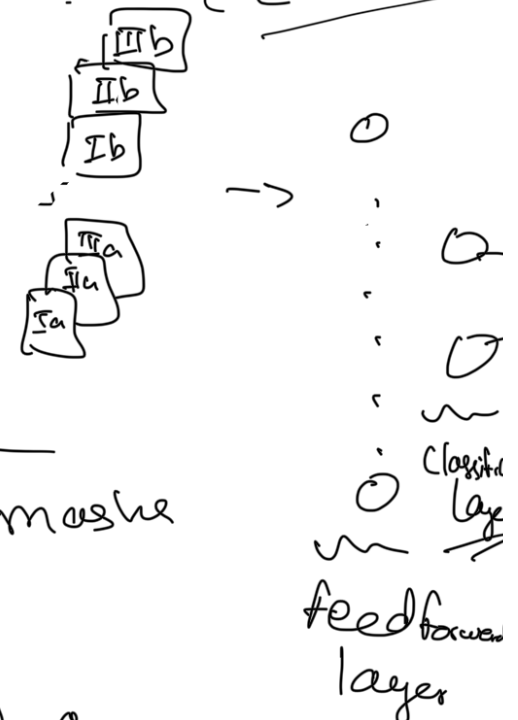
CNN for classification

LeNet

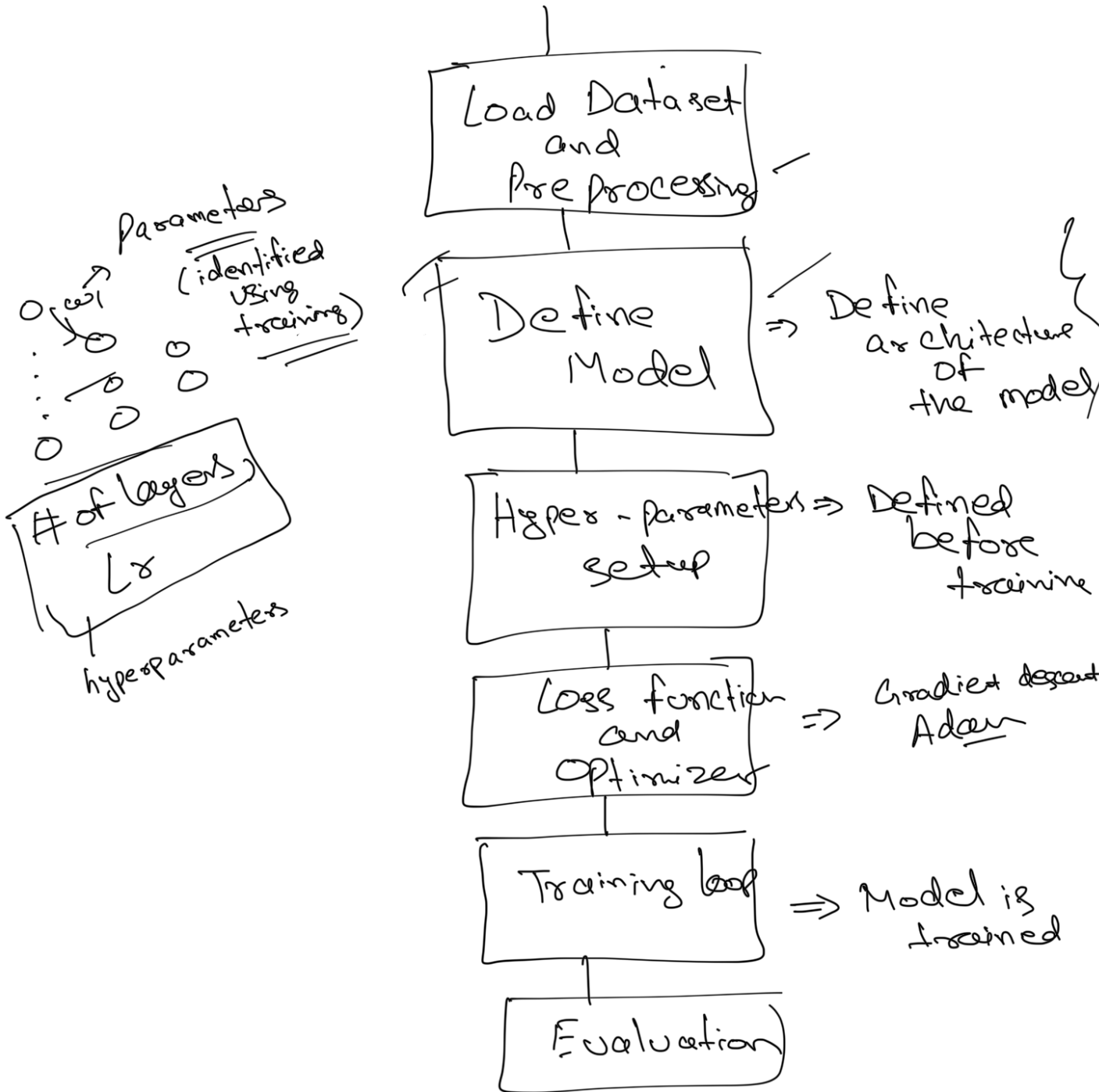
Algorithm for Deep learning Code

Template 1.1

Neural
(CNN)



+ implementation



Define Model

Pytorch $\Rightarrow$ pip install torch,
torchvision

How to define a model in Pytorch

Sequential



Model
is
defined
in
terms
of
layers
available
in
API

functional



Layers are
implemented
as
user defined
functions.



Model is
defined
in
terms of
these
function